09. Redux

Attention!

This training is **interview-driven**.

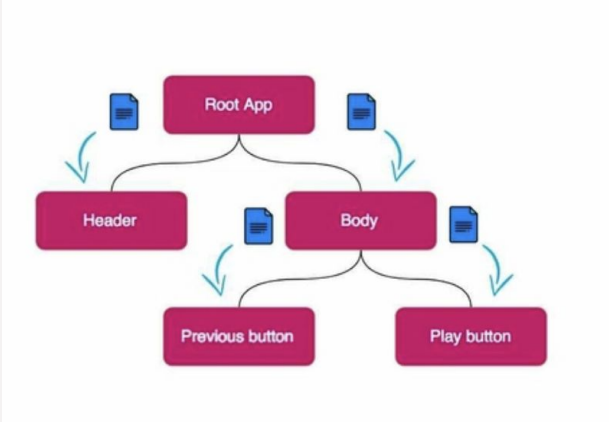
To become a qualified developer, *ChatGPT* and *YouTube* will always be your best friends.

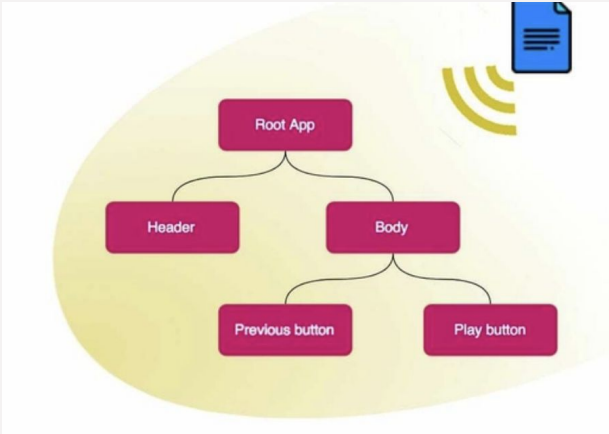
Outline

- Redux
- Redux Middleware
- Component Communication

Props drilling

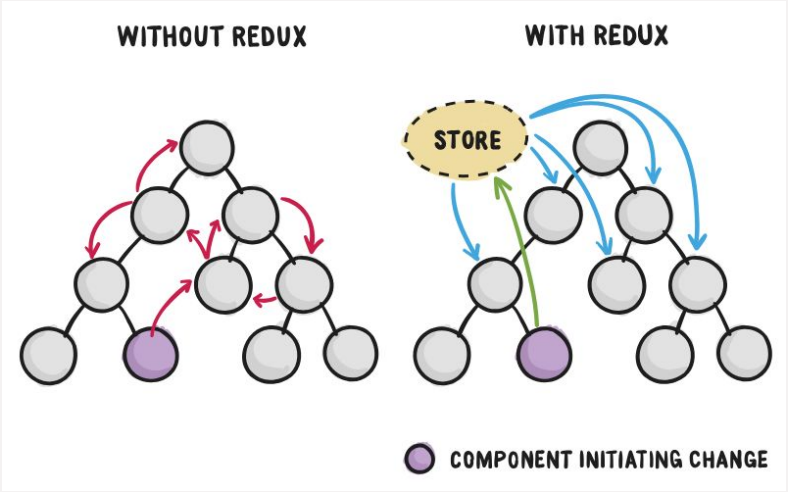
- **Props (properties)**: how data is passed from a parent component to a child component
- Props are read-only, one-way (from parent → child), and allow components to be reusable and dynamic
- **Props Drilling** happens when you have to pass props through **multiple** layers
    - Middle components don’t need it
    - Context API, State Management Libraries (Redux, Zustand)…

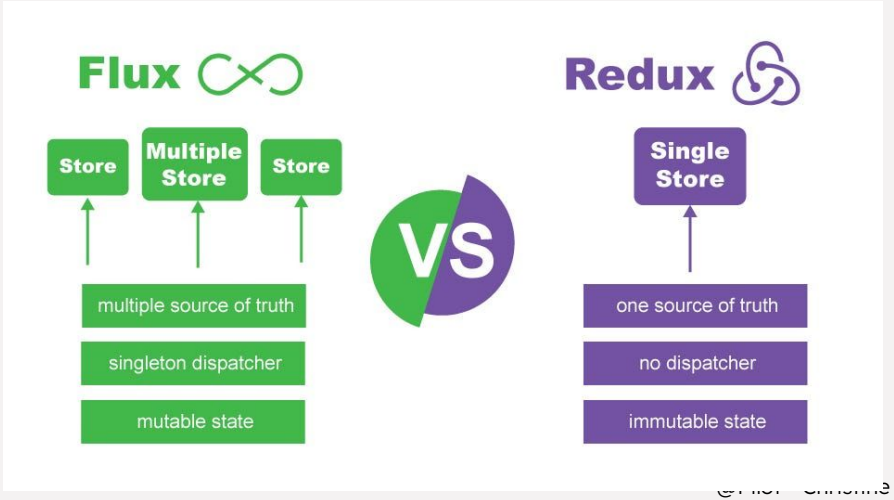




Redux

- Redux: an open-source **state management library** for centralizing the application state & logic
  - Inspired by Facebook's Flux architecture
  - Compatible with any frontend library/framework





Terminology

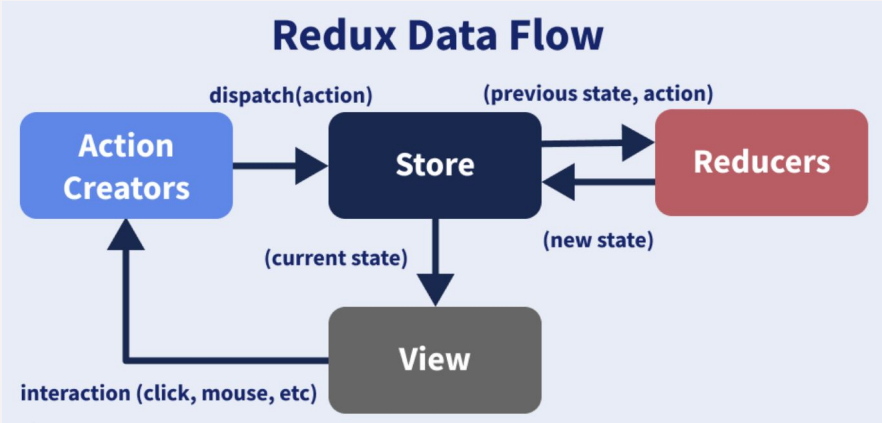
- Store: Holds the whole app's state
- State: app state object
- Action: A plain JS object that describes what happened
  - **type** is required: tells reducer what kind of change to make
  - **payload** is optional: provides data needed for the update
- Reducer: **Pure functions** that return the next state
  - Takes current state + action
  - Returns the new state

Redux Workflow

```
dispatch({ type: 'ADD_TO_CART', payload: { productId: 1, quantity: 2 } });
```

User interaction
↓
dispatch(action)
↓
store → reducer(state, action) → new state
↓
store updates its state
↓
UI re-renders

Add to cart



Redux Principles

- Single Source of Truth
  - The state of the entire application is stored in a single JS object (store)
    - app’s state is centralized
    - easier to manage, debug, and inspect
- State is Read-Only
  - The only way to change the state is to dispatch an action
  - Can’t change state directly
- Changes are Made with **Pure** Reducer **Functions**
  - Reducers take state + action
  - Return new state

@Pilot - Christine

Redux Toolkit (RTK)

- Redux Toolkit is the official, recommended way to write Redux logic
  - Simplifies Redux setup and code, reduces boilerplate
  - Generating actions + reducers together
  - Automatically includes useful middleware
  - Redux DevTools extension
  - Simplifies combining reducers

Concept	Traditional Redux	Redux Toolkit
Action creators	Manual	Auto-generated by `createSlice`
Reducers	Switch/case	Builder syntax or mutating style (via Immer)
Store	Manual + applyMiddleware	`configureStore` does it all
Async	Manual thunk/saga	`createAsyncThunk`

createSlice (RTK)

- Generates actions and reducers together
  - Internally, it uses createAction and createReducer
  - Uses **Immer** under the hood to produce immutable updates safely
- Automatically generates action type strings + action creator functions

```
const counterSlice = createSlice({
  name: 'counter',
  initialState,
  reducers: {
    increment(state) {
      state.value++
    },
    decrement(state) {
      state.value--
    },
    incrementByAmount(state, action: PayloadAction<number>) {
      state.value += action.payload
    },
  },
})

export const { increment, decrement, incrementByAmount } = counterSlice.actions
export default counterSlice.reducer
```

configureStore (RTK)

- A helper function provided by Redux Toolkit that simplifies the process of creating a Redux store
  - Creating the Redux store
  - Applying middleware (like Redux Thunk by default)
  - Combining reducers
  - Enabling Redux DevTools automatically in development

```
// before
const store = createStore(reducer);

// now
const store = configureStore({ reducer });
```

```
const store = configureStore({
  reducer: {},              // REQUIRED
  middleware: ...,          // OPTIONAL
  devTools: true/false,     // OPTIONAL
  preloadedState: {...},    // OPTIONAL
  enhancers: [...],         // RARELY USED
});
```

combineReducers (RTK)

- A utility function that combines multiple reducer functions into a single reducer
  - break reducer logic into smaller, focused reducers, each handling its piece of the state tree
  - Can still use combineReducers in RTK
    - configureStore can accept an object of reducers directly, no need for combineReducers manually

```
import { combineReducers, configureStore } from '@reduxjs/toolkit';
import userSlice from './userSlice';
import postsSlice from './postsSlice';

const rootReducer = combineReducers({
  user: userSlice.reducer,
  posts: postsSlice.reducer
});

const store = configureStore({
  reducer: rootReducer
});
```

```
const store = configureStore({
  reducer: {
    user: userSlice.reducer,
    posts: postsSlice.reducer
  }
});
```

createAsyncThunk (RTK)

- A Redux Toolkit helper for handling async logic that automatically generates pending, fulfilled, and rejected actions
- Parameters
  - type
  - payloadCreator
  - thunkAPI
- extraReducers
  - Async state is handled in extraReducers using status + error
- rejectWithValue
  - Allow custom error payloads instead of the default error object

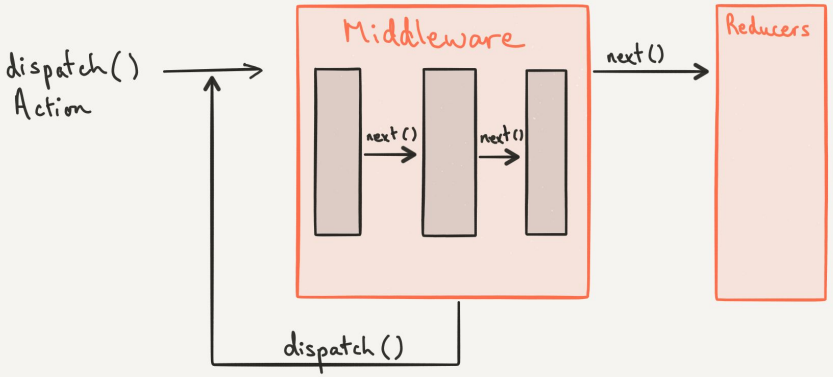
```
export const fetchUsers = createAsyncThunk(
  "users/fetchUsers",
  async (arg, thunkAPI) => {
    const response = await fetch("/api/users");
    return response.json();
  }
);
```

Redux in React

- react-redux: The official binding library for using Redux with React
  - **<Provider>**: Makes the Redux store available to all components down the tree
    - <Provider store={store}>
  - Hooks
    - **useSelector**: Read data from the store
      - const value = useSelector(state => state.someValue);
    - **useDispatch**: Trigger actions
      - const dispatch = useDispatch();
      - dispatch(action);

Redux Middleware

- Redux middleware is a layer between **dispatch(action)** and the **reducer**, used to handle **side effects**, async logic, logging…
- **Redux-Thunk**
  - dispatch **functions** (instead of just plain objects), enabling async logic
- **Redux Saga**
  - Redux Saga uses **generator functions** to write async logic that looks synchronous



Redux Thunk vs Redux Saga

Feature	Redux Thunk	Redux Saga
Async method	Functions	Generator functions (`function*`)
Complexity	Low	Medium–High
Setup	Minimal (built into Redux Toolkit)	More configuration needed
Suitable for	Simple async logic (API fetch)	Complex async flows (cancel, retry, chains)
Readability	Straightforward JS	Synchronous-like with generator syntax
Testing	Can be harder to test	Very testable due to generator structure
Control over effects	Limited	Fine-grained (debounce, cancel, retry, etc.)

Middleware (RTK)

- When using Redux Toolkit, thunk middleware is included by default
- Built-in middleware in RTK
  - redux-thunk
    - Async logic, API calls
  - Immutable State Check (development only)
  - Serializable State Check (development only)
- Customizing middleware in RTK
  - getDefaultMiddleware

```
configureStore({
  reducer,
  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware({
      serializableCheck: false,
      immutableCheck: false,
    }).concat(customMiddleware),
});
```

Redux Recap

[Define State Structure]

↓

[Create Slice(s) with createSlice()]

↓

[Configure Store with configureStore()]

↓

[Wrap App with <Provider store={store}>]

↓

[Use useSelector to read state]

[Use useDispatch to update state]

Redux: Pros and Cons

- **When**
  - Your app has complex state shared across many components
  - You need predictable, testable state transitions
  - Your app requires powerful debugging and time travel
  - You're handling complex async logic or side effects
- **When not**
  - Your app is small (e.g. a form, a few components)
  - Local component state or React Context would do the job
  - You want minimal setup / less boilerplate

Redux vs Context API

	Redux	**Context API**
Application complexity	Large-scale, complex states, frequent updates	Small-scale, simple states, infrequent updates
Difficulty	Complex, abstract	Easy
Native	Third-party	Built-in
Testing	Pure reducers	
Maintainability	Centralized state/logic Strict structure	
Debugging	Redux dev tools	

Component Communication

- **Parent → Child (props)**
  - Props are read-only
- **Child → Parent (callback props)**
  - Children communicate upward by calling functions passed via props
- **Sibling → Sibling (lift state up)**
  - State should be lifted to the lowest common ancestor
- **Context API**
  - Avoid prop drilling, Share simple global data
- **Redux**
  - Components no longer talk to each other directly, they communicate through the store

Additional materials

- Flux architecture
- PubSub(Publish-Subscribe)
- Redux vs Zustand / MobX

Homework - Requirement

- 全程recording: 八股闭眼 & 摘耳机, 无AI辅助
- 全程recording: Coding可适当AI辅助, 但需写完(边解释边写), 模拟真实面试情景
- 录音提交地址: https://drive.google.com/drive/folders/1Mrm341uOIS8c2Ty6NwYvvSybHa6hESem?usp=drive_link
- 提交格式: 小组+日期, 例如Group1_12/07/2025
- 提交截止时间: due第二天5PM (周四作业递延到下周一)

Homework - Class Code

- Write the code as taught in class, take a screenshot of your code, and post it in the Wechat group. **It's due the same night.**

Homework - Short Answer Questions

1. Why can’t reducers handle async logic or side effects?
2. What is redux-thunk?
3. Why does Redux-Saga use generators?
4. What is createAsyncThunk?
5. How would two sibling components communicate through props?
6. What is Redux?
7. What are the core principles of Redux?
8. Explain what the Redux store, actions, reducers are and what they do. **What is the workflow?**
9. explain useSelector and useDispatch
10. How do you create/configure a store in redux?

@Pilot - Christine

Homework - Short Answer Questions

11. What are the advantages of redux?
12. When would you use the Context API? How about Redux?
13. What is the benefit of Redux Dev Tools?
14. What is react-redux? How does it differ from the redux library?
15. How do you asynchronously dispatch an action?
16. How to access Redux store outside a component?
17. How to reset state in Redux?
18. What is the proper way to access Redux store?
19. What is redux-saga?
20. What is the mental model of redux-saga?

Homework - Short Answer Questions

21. What is Redux Thunk?
22. What are the differences between redux-saga and redux-thunk
23. What are Redux selectors and Why to use them?
24. How to add multiple middlewares to Redux?
25. Have you built any reusable components? What are best practices?
26. What is Lifting State Up in React?
27. what is the typical Redux implementation flow?

Homework - Coding Questions

1. Build a Simple Todo App Using React + Redux Toolkit. Features:
    a. Add a todo item
    b. Delete a todo item
    c. Toggle a todo item as completed